

Elastic Topology Management for Hybrid Retrieval-Augmented Generation: A Measurement Study across Edge, LAN, and WAN

M. Abdullah Ali

M. Abdullah Aamir

Department of Data Science, FAST-NUCES — Proposed supervisor: [to be confirmed]

September 14, 2026

1 Background and Current State

We have built and deployed a working *hybrid* Retrieval-Augmented Generation (RAG) system: sparse keyword retrieval (BM25 via Tantivy), dense semantic retrieval (BGE-M3 embeddings searched in Qdrant), reciprocal-rank fusion, and LLM generation (Llama-3-8B-Instruct, AWQ 4-bit, served by vLLM [14, 13]). The corpus is the full MS MARCO passage collection [12]: **8,841,823 passages** embedded at 1024 dimensions, HNSW-indexed in Qdrant, ingested at roughly 538 passages per second on an RTX 3090. The system runs on two LAN machines over gRPC (WireGuard): a gateway node (embedding, dense+sparse retrieval, fusion) on an Intel i7-10750H laptop with GTX 1660 Ti, and a generation node (document lookup, vLLM) on an RTX 3090 host. An earlier three-node version placed a third, low-power laptop (Intel CPU, 16 GB RAM) at the network edge as entry point and sparse-retrieval engine, across a metropolitan-fiber link treated as the WAN leg.

2 A Failed Hypothesis That Points at a Real Gap

The original design hypothesized a 30–40% TTFT (time-to-first-token) reduction by running edge-side sparse retrieval in parallel with transit of the query toward the LAN nodes: $T_{\text{retrieval}} = \max(T_{\text{sparse}}, T_{\text{WAN}} + T_{\text{dense}})$. Two things undid this. First, the “WAN” was not much of one: the edge laptop sat at a home across the city on metropolitan fiber, and our benchmarks put the round trip at only 5–15 ms (20 ms with WireGuard and protocol overhead) — there was almost no transit latency to hide. Second, and more fundamentally, parallel execution can save at most $\min(T_{\text{sparse}}, T_{\text{WAN}} + T_{\text{dense}})$, and our 100-query logs from the three-node run show sparse retrieval took just 0.6–10 ms — too cheap to be worth distributing — while the split made the dense branch pay the extra 20 ms plus coordination overhead. When the edge laptop then failed outright, we consolidated its duties into the gateway, and the two-node system worked fine.

That failure motivates this proposal: placement by intuition silently costs performance, the stages worth distributing are the *expensive* ones (dense retrieval, document handling, generation — not sparse lookup), and topology change is a normal event in real deployments — yet the literature, as far as we can find, cannot answer: *where should each stage of a hybrid RAG pipeline run, under which network conditions, and what exactly do we lose when the topology degrades?*

3 The Gap in the Literature

DNN partitioning between device and cloud is well studied at the *layer* level (Neurosurgeon [1], PipeEdge [2]); LLM serving splits model *phases* across machines (Splitwise [3]). For RAG specifically, existing work optimizes KV caching of retrieved knowledge (RAGCache [4], TurboRAG [5]), serving configuration on a *single* server (RAGO [6], METIS [7]), index placement on one device (EdgeRAG [8], VectorLiteRAG [9]), or data distribution across edge/cloud tiers (EACO-RAG [10], DGRAG [11]). We could not find a measurement study and placement policy for *which pipeline stage runs on which machine* in a multi-node hybrid RAG deployment: the five stages (sparse, dense, fusion, hydration, generation) can each sit on a different node, and the right answer changes with RTT, bandwidth, and node capability. Topology failure (what happens to latency and answer quality when a node drops out and roles consolidate) appears entirely unmeasured for RAG.

4 Research Questions

- **RQ1 (Placement).** Which node placement for each pipeline stage minimizes TTFT and end-to-end latency under a given network regime (RTT, bandwidth), and where does the best placement change?
- **RQ2 (Degradation).** When a node fails and its roles consolidate (our real three-to-two-node event), what is the cost in latency and retrieval quality, and how fast can the system adapt?
- **RQ3 (Policy).** Can a simple analytical cost model — per-stage compute times, payload sizes, network terms — predict the best placement well enough to drive decisions without the full experiment matrix?

5 Method

Testbed. The existing heterogeneous nodes (RTX 3090 host, GTX 1660 Ti laptop, revived low-power laptop) over WireGuard; the full 8.84M-passage index is already in place. **Instrumentation.** We will extend the existing per-stage timing logs into a structured benchmark harness (per-stage milliseconds, TTFT, decode throughput, true stream end-to-end) and add retrieval-quality evaluation against MS MARCO qrels (recall@ k), which the system does not yet measure; the sparse index will be scaled from a small test index to the full corpus. **Network control.** Per-link netem sweeps of delay and bandwidth (e.g. 0/10/40/80 ms RTT) make the WAN regime reproducible; key configurations are re-validated over the genuine WAN link to the edge laptop. **Placement matrix.** Stage-to-node assignment becomes a configuration switch (fusion on gateway vs. generator; hydration local vs. remote; sparse retrieval at the edge vs. LAN), reconstructing the archived three-node code paths. **Campaign.** A factorial sweep over placement $\times$ network regime $\times$ retrieval depth with repeats and percentile statistics; a degradation experiment kills nodes mid-load to measure failover cost; the RQ3 cost model is fit on half the configurations and validated on the held-out half.

6 Expected Contributions

(1) The first factorial measurement of stage-to-node placement for hybrid RAG on commodity heterogeneous hardware under controlled WAN regimes; (2) a quantified characterization of topology degradation, grounded in a real failure event; (3) a validated cost-model-based placement policy; (4) an open, reproducible testbed with all artifacts.

7 Timeline and Target Venues

Weeks 1–2: harness, full-corpus sparse index, quality evaluation. Week 3: placement matrix and network emulation. Weeks 4–5: campaigns, cost model, analysis. Weeks 6–8: writing, internal review, submission. We target Q1 journals with rolling submission: *Future Generation Computer Systems*, *Computer Networks*, or *IEEE Transactions on Cloud Computing* (all currently Q1 in SCImago/JCR rankings). A conference extension (e.g. IEEE ICDCS 2027, call expected around December 2026) remains possible for the policy contribution.

References

- [1] Y. Kang et al., “Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge,” *Proc. ASPLOS*, 2017.
- [2] Y. Hu et al., “PipeEdge: Pipeline Parallelism for Large-Scale Model Inference on Heterogeneous Edge Devices,” *Proc. DSD*, 2022.
- [3] P. Patel et al., “Splitwise: Efficient Generative LLM Inference Using Phase Splitting,” *Proc. ISCA*, 2024.
- [4] C. Jin et al., “RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation,” arXiv:2404.12457, 2024.
- [5] S. Lu et al., “TurboRAG: Accelerating Retrieval-Augmented Generation with Precomputed KV Caches for Chunked Text,” *Proc. EMNLP*, 2025.
- [6] W. Jiang et al., “RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving,” *Proc. ASPLOS*, 2025.
- [7] S. Ray et al., “METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation,” arXiv:2412.10543, 2025.
- [8] K. Seemakhupt, S. Liu, and S. Khan, “EdgeRAG: Online-Indexed RAG for Edge Devices,” arXiv:2412.21023, 2024.
- [9] J. Kim and D. Mahajan, “VectorLiteRAG: Latency-Aware and Fine-Grained Resource Partitioning for Efficient RAG,” arXiv:2504.08930, 2025.
- [10] J. Li et al., “EACO-RAG: Towards Distributed Tiered LLM Deployment Using Edge-Assisted and Collaborative RAG with Adaptive Knowledge Update,” arXiv:2410.20299, 2024.
- [11] W. Zhou, Y. Yan, and Q. Yang, “DGRAG: Distributed Graph-based Retrieval-Augmented Generation in Edge-Cloud Systems,” arXiv:2505.19847, 2025.
- [12] P. Bajaj et al., “MS MARCO: A Human Generated MACHine Reading COMprehension Dataset,” arXiv:1611.09268, 2016.
- [13] J. Chen et al., “M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation,” arXiv:2402.03216, 2024.
- [14] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *Proc. SOSP*, 2023.